[Year]

Distributed file system

12TH PROJECT

HABIBA ABORAYA, 211002226

This project implements a distributed file system using Java RMI, where multiple nodes cooperate to perform file operations in a consistent and ordered manner. Each node can initiate file operations such as upload, delete, search, and download, while communication between nodes is handled through remote method calls. To ensure consistency across all nodes, the system uses a Totally Ordered Multicast mechanism based on Lamport logical clocks and acknowledgment messages. This guarantees that all file operation requests are executed in the same order on every node, even when requests are issued concurrently.

My packages are:

1. common
    - Request
    - ACK
2. rmi
    - NodeInterface
3. node
    - Node
4. client
    - StartSystem

1. **common**
   - **Request**
     The Request class represents a file operation request in the distributed file system. It is sent between nodes using Java RMI, so it implements Serializable. Each request contains the type of operation (upload, delete, search, or download), the name of the file, the logical timestamp, the ID of the node that sent the request, and a unique request ID. The timestamp and request ID are used to ensure that all nodes execute requests in the same order as part of the Totally Ordered Multicast mechanism.

```java
package common;

import java.io.Serializable;

public class Request implements Serializable {

    // Types of operations supported by the system
    public enum Type {
        UPLOAD,
        DELETE,
        DOWNLOAD,
        SEARCH
    }

    private Type type;          // Operation type
    private String fileName;    // Target file
    private int timestamp;      // Logical clock value
    private int senderId;       // Node that sent the request
    private int requestId;      // Tie-breaker for equal timestamps

    public Request(Type type, String fileName, int timestamp, int senderId, int requestId)
{
        this.type = type;
        this.fileName = fileName;
        this.timestamp = timestamp;
        this.senderId = senderId;
        this.requestId = requestId;
    }

    public Type getType() {
        return type;
    }

    public String getFileName() {
        return fileName;
    }

    public int getTimestamp() {
        return timestamp;
    }
```

```
    public int getSenderId() {
        return senderId;
    }

    public int getRequestId() {
        return requestId;
    }
}
```

- ACK

The ACK class represents an acknowledgment message sent between nodes. It is used to confirm that a node has received a specific request. Each acknowledgment contains the request ID and the ID of the node sending the acknowledgment. A request is executed only after acknowledgments are received from all nodes, which ensures correct ordering and consistency as part of the Totally Ordered Multicast protocol.

```
package common;

import java.io.Serializable;

public class ACK implements Serializable {

    private int requestId;
    private int senderId;

    public ACK(int requestId, int senderId) {
        this.requestId = requestId;
        this.senderId = senderId;
    }

    public int getRequestId() {
        return requestId;
    }

    public int getSenderId() {
        return senderId;
    }
}
```

2. rmi
    - NodeInterface

The NodeInterface defines the remote methods that allow nodes to communicate using Java RMI. It extends the Remote interface and declares methods for receiving file operation requests and acknowledgment messages. These methods enable nodes to exchange requests and ACKs through remote method calls, allowing the distributed system to coordinate operations and enforce totally ordered multicast.

```
package rmi;

import common.Request;
import common.ACK;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface NodeInterface extends Remote {

    // Receive a request from another node
    void receiveRequest(Request request) throws RemoteException;

    // Receive an ACK from another node
    void receiveACK(ACK ack) throws RemoteException;
}
```

3. node
    - Node

The Node class represents a single node in the distributed file system and contains the core logic of the system. Each node communicates with other nodes using Java RMI and maintains a Lamport logical clock to order requests. Nodes store requests in a priority queue and execute them only after receiving acknowledgments from all other nodes, which guarantees Totally Ordered Multicast. Upload and delete operations are executed by all nodes, while search and download operations are handled only by the leader node to avoid duplicate responses.

```
package node;

import common.Request;
import common.ACK;
import rmi.NodeInterface;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.io.File;
import java.util.*;


public class Node extends UnicastRemoteObject implements
NodeInterface {

    // Node identity
    private int nodeId;
```

```java
    private int leaderId;
    private int totalNodes;

    // Lamport logical clock
    private int logicalClock;

    // Directory for this node's files
    private File nodeDirectory;

    // Request queue ordered by timestamp then requestId
    private PriorityQueue<Request> requestQueue;

    // requestId -> set of nodeIds that acknowledged
    private Map<Integer, Set<Integer>> ackMap;

    // References to all nodes (for multicast)
    private Map<Integer, NodeInterface> nodes;

    // Constructor
    public Node(int nodeId, int totalNodes, int leaderId) throws
RemoteException {
        super();
        this.nodeId = nodeId;
        this.totalNodes = totalNodes;
        this.leaderId = leaderId;
        this.logicalClock = 0;

        // Create node directory
        this.nodeDirectory = new File("files/node" + nodeId);
        if (!nodeDirectory.exists()) {
            nodeDirectory.mkdirs();
        }

        // Initialize priority queue with total ordering
        this.requestQueue = new PriorityQueue<>(
                (r1, r2) -> {
                    if (r1.getTimestamp() != r2.getTimestamp()) {
                        return Integer.compare(r1.getTimestamp(),
r2.getTimestamp());
                    }
                    return Integer.compare(r1.getRequestId(),
r2.getRequestId());
                }
        );

        this.ackMap = new HashMap<>();
        this.nodes = new HashMap<>();
    }

    // Register other nodes (called during startup)
    public void addNode(int id, NodeInterface node) {
        nodes.put(id, node);
    }

    //REQUEST SENDING (MULTICAST)

    public synchronized void multicastRequest(Request.Type type,
```

```
String fileName)
            throws RemoteException {

        // Increment logical clock before sending
        logicalClock++;

        // Generate unique request ID
        int requestId = logicalClock * 100 + nodeId;

        // Create request
        Request request = new Request(
                type,
                fileName,
                logicalClock,
                nodeId,
                requestId
        );

        // Multicast request to all nodes (including self)
        for (NodeInterface targetNode : nodes.values()) {
            targetNode.receiveRequest(request);
        }
    }

    // ---------------- RMI METHODS ----------------

    @Override
    public synchronized void receiveRequest(Request request)
throws RemoteException {

        // Update logical clock on receive
        logicalClock = Math.max(logicalClock,
request.getTimestamp()) + 1;

        // Add request to the priority queue
        requestQueue.add(request);

        // Initialize ACK tracking
        ackMap.putIfAbsent(request.getRequestId(), new
HashSet<>());

        // Multicast ACK to all nodes
        for (NodeInterface targetNode : nodes.values()) {
            ACK ack = new ACK(request.getRequestId(), nodeId);
            targetNode.receiveACK(ack);
        }
    }


    @Override
    public synchronized void receiveACK(ACK ack) throws
RemoteException {

        int requestId = ack.getRequestId();
        int senderId = ack.getSenderId();

        // Record ACK
```

```
        ackMap.putIfAbsent(requestId, new HashSet<>());
        ackMap.get(requestId).add(senderId);

        // Check if requests can now be executed
        tryExecuteRequests();
    }

    // ---------------- EXECUTION LOGIC ----------------


    private void tryExecuteRequests() {

        while (!requestQueue.isEmpty()) {

            Request request = requestQueue.peek();
            int requestId = request.getRequestId();

            // Check if all nodes acknowledged this request
            Set<Integer> ackedNodes = ackMap.get(requestId);
            if (ackedNodes == null || ackedNodes.size() <
totalNodes) {
                return;
            }

            // Safe to execute
            requestQueue.poll();
            ackMap.remove(requestId);

            executeRequest(request);
        }
    }


    private void executeRequest(Request request) {

        File targetFile = new File(nodeDirectory,
request.getFileName());

        switch (request.getType()) {

            case UPLOAD:
                try {
                    if (!targetFile.exists()) {
                        targetFile.createNewFile();
                        System.out.println("Node " + nodeId +
                                " UPLOADED " +
request.getFileName());
                    } else {
                        System.out.println("Node " + nodeId +
                                " already has " +
request.getFileName());
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                }
                break;
```

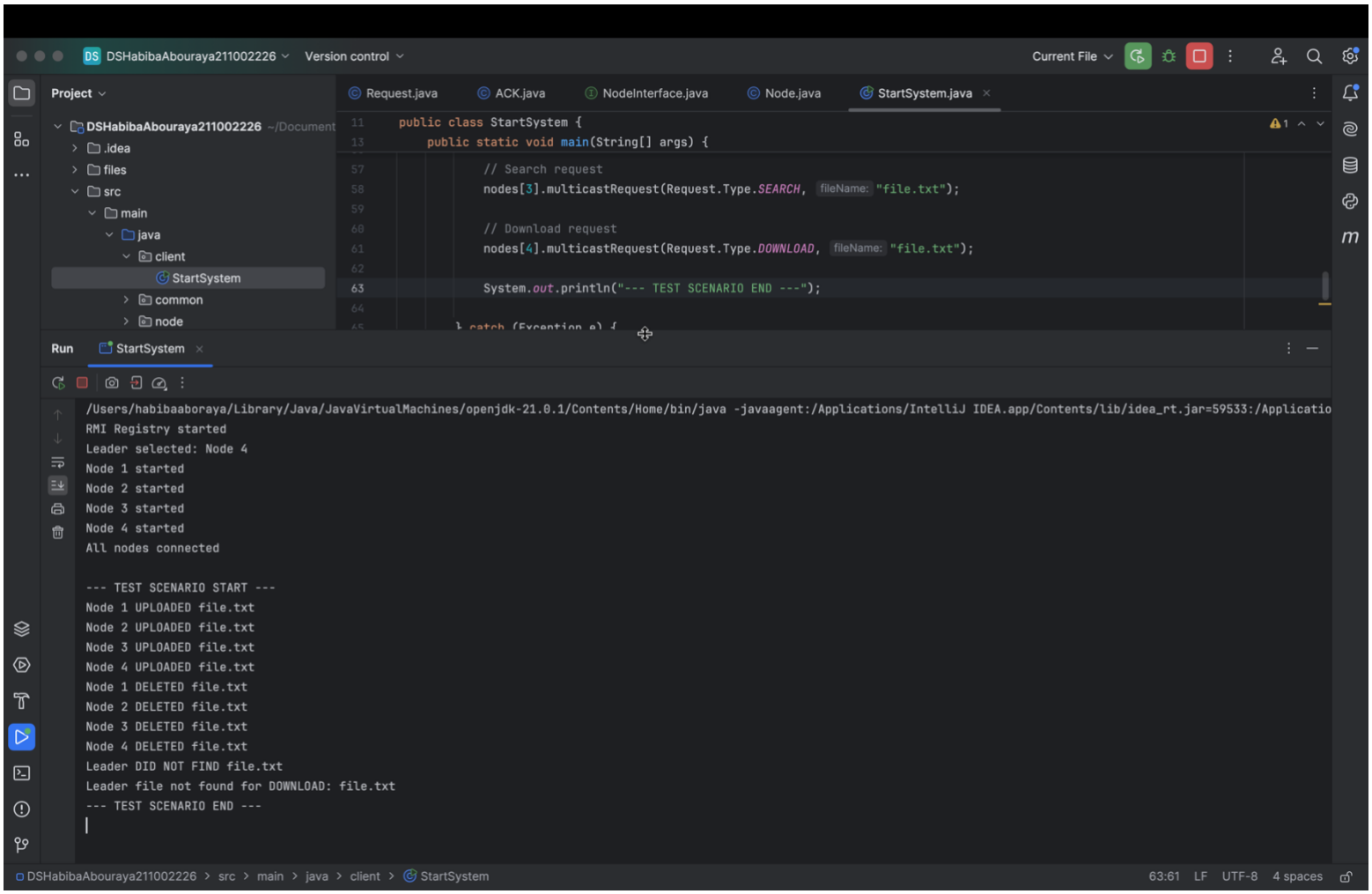
```
                case DELETE:
                    if (targetFile.exists()) {
                        targetFile.delete();
                        System.out.println("Node " + nodeId +
                                " DELETED " + request.getFileName());
                    } else {
                        System.out.println("Node " + nodeId +
                                " file not found for DELETE: " +
request.getFileName());
                    }
                    break;

                case SEARCH:
                    if (nodeId == leaderId) {
                        if (targetFile.exists()) {
                            System.out.println("Leader FOUND " +
request.getFileName());
                        } else {
                            System.out.println("Leader DID NOT FIND "
+ request.getFileName());
                        }
                    }
                    break;

                case DOWNLOAD:
                    if (nodeId == leaderId) {
                        if (targetFile.exists()) {
                            System.out.println("Leader DOWNLOADING "
+ request.getFileName());
                        } else {
                            System.out.println("Leader file not found
for DOWNLOAD: "
                                    + request.getFileName());
                        }
                    }
                    break;
            }
        }
    }
```

4. client
    - StartSystem

The StartSystem class is responsible for starting and testing the distributed file system. It initializes the RMI registry, creates four nodes, and randomly selects a leader node. Each node is registered in the RMI registry and connected to all other nodes using remote references. After initialization, this class runs test scenarios that include upload, delete, search, and download requests to demonstrate the correctness of Totally Ordered Multicast and the supported file operations.

```
package client;

import node.Node;
import rmi.NodeInterface;
import common.Request;
```

```java
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Random;

public class StartSystem {

    public static void main(String[] args) {

        try {
            int totalNodes = 4;

            // 1. Start RMI Registry
            Registry registry =
LocateRegistry.createRegistry(1099);
            System.out.println("RMI Registry started");

            // 2. Select leader randomly
            Random random = new Random();
            int leaderId = random.nextInt(totalNodes) + 1;
            System.out.println("Leader selected: Node " +
leaderId);

            // 3. Create nodes
            Node[] nodes = new Node[totalNodes + 1];

            for (int i = 1; i <= totalNodes; i++) {
                nodes[i] = new Node(i, totalNodes, leaderId);
                registry.rebind("Node" + i, nodes[i]);
                System.out.println("Node " + i + " started");
            }

            // 4. Connect nodes together
            for (int i = 1; i <= totalNodes; i++) {
                for (int j = 1; j <= totalNodes; j++) {
                    NodeInterface stub =
                            (NodeInterface)
registry.lookup("Node" + j);
                    nodes[i].addNode(j, stub);
                }
            }

            System.out.println("All nodes connected");

            // 5. TEST SCENARIO (important for grading)

            System.out.println("\n--- TEST SCENARIO START ---");

            // Upload from Node 1
            nodes[1].multicastRequest(Request.Type.UPLOAD,
"file.txt");

            // Concurrent delete from Node 2
            nodes[2].multicastRequest(Request.Type.DELETE,
"file.txt");

            // Search request
```

```
                nodes[3].multicastRequest(Request.Type.SEARCH,
"file.txt");

                // Download request
                nodes[4].multicastRequest(Request.Type.DOWNLOAD,
"file.txt");


            } catch (Exception e) {
                e.printStackTrace();
            }
        }
```

5. Output